

HyphaeDB: A Living Knowledge Topology for Agent-First Memory

Using HNSW Graph Structure as a Gossip-Based Communication Fabric
for Multi-Agent Knowledge Propagation and Emergent Coordination

Krishna Halaharvi

HyphaeDB

hello@hyphaedb.com

<https://github.com/hyphae-db>

March 2026

Abstract

Every existing vector database and agent memory framework treats memory as passive storage that agents query explicitly. No system propagates knowledge between agents through the memory layer itself. We introduce **HyphaeDB**, an agent-native memory infrastructure that reinterprets the Hierarchical Navigable Small World (HNSW) graph topology—the data structure at the core of every modern vector database—not as a search optimization, but as a *communication fabric* for multi-agent AI systems. In HyphaeDB, agents are nodes in the vector space with persistent positions, knowledge propagates via a gossip protocol through the graph’s neighbor structure with energy-based attenuation, and emergent behaviors—contradiction detection, pattern crystallization, and consensus formation—arise from the combination of topology, propagation dynamics, and local interaction rules. We present the architecture built on three primitives (knowledge nodes, topology edges, and memory diffs), a multi-layer abstraction hierarchy with promotion via emergent consensus, and theoretical analysis grounding the system in small-world network theory, epidemic broadcast protocols, and swarm intelligence. We provide a reference implementation on PostgreSQL with pgvector and describe a concrete deployment in Swarm-Driven Development, a multi-agent software engineering methodology. HyphaeDB represents, to our knowledge, the first system to combine navigable small-world topology with gossip-based knowledge propagation for multi-agent coordination.

Keywords: multi-agent systems, agent memory, HNSW, gossip protocols, knowledge propagation, swarm intelligence, vector databases, emergent behavior

1 Introduction

The rapid adoption of AI agent systems—projected to grow from \$7.5 billion in 2025 to over \$50 billion by 2030 [MarketsandMarkets, 2025, Grand View Research, 2025]—has created urgent demand for infrastructure that supports not just individual agent intelligence but *collective* agent intelligence. As organizations deploy an average of 12 agents per enterprise [Salesforce, 2026] and Gartner predicts 40% of enterprise applications will embed task-specific AI agents by end of 2026 [Gartner, 2025], the infrastructure gap for multi-agent coordination has become a critical bottleneck.

At the center of this gap is memory. Every existing agent memory framework—Mem0 [Yadav et al., 2025], Letta [Packer et al., 2023], LangMem [LangChain, 2025], and others—implements the same fundamental pattern: extract knowledge from agent interactions, store it in a vector database or graph, and retrieve it when an agent queries for it. Memory is passive. Between writes and reads, the storage layer is inert. It has no mechanism to propagate knowledge between agents, no awareness of which agents need what information, and no capacity for emergent behavior.

This paper introduces **HyphaeDB**, a fundamentally different primitive for agent memory. HyphaeDB reinterprets the HNSW (Hierarchical Navigable Small World) graph [Malkov and Yashunin, 2020]—the index structure used by every major vector database for approximate nearest neighbor search—as a *communication fabric* where:

1. **Agents are nodes** in the vector space with persistent positions that drift based on their work.
2. **Knowledge propagates** via a gossip protocol through the graph’s neighbor structure, with an energy model that attenuates based on relevance and importance.
3. **Emergent behaviors** arise from the topology: contradiction detection, pattern crystallization, and consensus formation occur without explicit programming.

The result is a system where memory does not sit in storage waiting to be found. It flows through a living topology, finding the agents that need it, surfacing conflicts before they become errors, and crystallizing patterns that no single agent could observe in isolation.

Contributions. This paper makes four contributions:

1. We identify the reinterpretation of HNSW graph topology as a communication fabric for multi-agent knowledge propagation—to our knowledge, the first proposal to use navigable small-world graphs for this purpose.
2. We design a complete gossip protocol adapted for semantic knowledge propagation, including an energy-based attenuation model, multi-layer abstraction with promotion via emergent consensus, and standing beacon subscriptions.
3. We provide theoretical analysis grounding the system’s properties in small-world network theory, epidemic broadcast models, and swarm intelligence.
4. We present a reference implementation on PostgreSQL with pgvector and describe deployment in a multi-agent software development workflow.

2 Related Work

2.1 Agent Memory Systems

The agent memory landscape has evolved rapidly since 2023. **MemGPT/Letta** [Packer et al., 2023] pioneered an OS-inspired memory hierarchy—core memory (in-context, analogous to RAM), recall memory (conversation history, analogous to disk), and archival memory (external databases). Its 2025 innovation of “sleep-time compute” separated memory management from active conversation, allowing asynchronous memory refinement. **Mem0** [Yadav et al., 2025] implements a two-phase memory pipeline (extraction then update) with an LLM reconciling candidate memories against a vector store, achieving 26% higher accuracy than OpenAI’s memory on the LOCOMO benchmark. Its graph variant (Mem0^g) uses directed labeled graphs for entity-relation storage. **LangMem** [LangChain, 2025] provides memory tools integrated with LangGraph’s storage layer, supporting semantic, episodic, and procedural memory types.

All these systems share a fundamental limitation: memory is something agents *read from* and *write to*. No

system propagates knowledge between agents through the memory layer itself. A 2025 survey on LLM-based multi-agent system memory [Zhang et al., 2025] confirms that “MAS memory must support coordination across multiple contextual layers,” but identifies no production system achieving this. The A-Mem system [Xu et al., 2025] introduces Zettelkasten-inspired dynamic memory organization where memories self-link, representing the closest academic work to active memory, but operates within a single agent.

2.2 HNSW and Navigable Small-World Graphs

The HNSW algorithm [Malkov and Yashunin, 2020] constructs a multi-layer proximity graph where Layer 0 contains all data points with dense local connections, and upper layers contain progressively sparser subsets with longer-range links. Layer assignment follows an exponential distribution: $\ell(x) = \lfloor -\ln(\text{uniform}(0, 1)) \cdot m_L \rfloor$. The structure inherits the small-world properties identified by Watts and Strogatz [1998]—high local clustering combined with short average path lengths scaling as $O(\log N)$ —while satisfying the navigability condition proven by Kleinberg [2000]: greedy routing achieves $O(\log^2 N)$ delivery time when long-range connections follow $P(u \rightarrow v) \propto d(u, v)^{-d}$. Malkov and Ponomarenko [2016] demonstrated that HNSW achieves this distribution emergently through incremental construction.

The 2024 Hub Highway Hypothesis [Munyampirwa et al., 2024] revealed that HNSW graphs naturally develop well-connected hub nodes serving as routing shortcuts, structurally analogous to backbone routers in communication networks. These hubs form without explicit design, emerging from the construction process.

No published research uses HNSW topology for communication routing or agent coordination. However, precedents exist in adjacent domains: Symphony [Manku et al., 2003] built a DHT on Kleinberg’s navigable small-world model; Freenet’s darknet [Clarke et al., 2005] used small-world routing for anonymous communication; skip graphs [Aspnes and Shah, 2003] generalized the multi-layer structure to distributed settings.

2.3 Gossip Protocols

The foundational work by Demers et al. [1987] established two complementary gossip mechanisms: *anti-entropy* (full state synchronization) and *rumor mongering* (probabilistic spreading). The SI epidemic model governing gossip follows logistic growth: starting from one informed node in N , approximately $N \cdot (1 - e^{-fk/N})$ nodes are reached after k rounds with fanout f , achieving full propagation in $O(\log N)$ rounds with high probability.

SWIM [Das et al., 2002] achieves $O(N)$ message complexity by piggybacking updates on probe/ack messages. HyParView [Leitão et al., 2007a] maintains dual partial views providing fault tolerance under 80–95% node failure. Plumtree [Leitão et al., 2007b] constructs self-organizing broadcast trees achieving near-optimal $N - 1$ payload messages per broadcast. The T-Man protocol [Jelasity and Babaoglu, 2009] uses gossip exchanges to construct arbitrary overlay topologies with logarithmic convergence.

Spatial gossip [Kempe et al., 2001] analyzed gossip where peer selection probability is inversely proportional to distance, achieving $O(\log^{1+\varepsilon} N)$ delivery at the navigable sweet spot—directly applicable to gossip over HNSW’s distance-aware topology. A 2025 vision paper [Bénézit et al., 2025] explicitly proposes gossip as a first-class primitive for agentic multi-agent systems but specifies no topology. HyphaeDB provides the topology.

2.4 Swarm Intelligence and Emergence

Reynolds’ Boids model [Reynolds, 1987] demonstrated that three local rules produce coherent flocking without central control. Ant colony optimization [Dorigo and Stützle, 2004] showed that local pheromone dynamics produce globally optimal paths. Grassé’s stigmergy [Grassé, 1959] established that complex coordinated activity can occur without planning, communication, or mutual awareness—precisely the kind of coordination HyphaeDB enables digitally.

Olfati-Saber et al. [2007] showed that network topology directly affects consensus dynamics through the eigenvalues of the graph Laplacian, with small-world properties enabling fast convergence. Moore and Newman [2000] demonstrated that epidemic spreading on small-world networks is dramatically accelerated: high clustering ensures local transmission while short paths enable rapid system-wide propagation. Buehler [2025] showed that recursive graph expansion produces scale-free networks with emergent conceptual hubs and bounded diameter growth, exhibiting “human-like hierarchical knowledge formation.”

3 Architecture

HyphaeDB is built on three primitives. All system functionality—knowledge storage, agent communication, subscription management, consensus formation—reduces to compositions of these constructs.

3.1 Primitive 1: Knowledge Nodes

Every entity in the topology is a *knowledge node* with a position in vector space:

$$\text{node} = (\text{id}, \tau, \mathbf{e}, \ell, \phi) \quad (1)$$

where $\tau \in \{\text{cell}, \text{agent}, \text{scene}, \text{beacon}\}$ is the node type, $\mathbf{e} \in \mathbb{R}^d$ is the embedding vector ($d = 1536$ by default), $\ell \in \{0, 1, 2\}$ is the layer assignment, and ϕ is a type-specific payload.

Cell nodes represent atomic knowledge units: decisions, facts, patterns, risks, lessons, constraints, and other typed information. Each cell has a salience score $s \in [0, 1]$ indicating importance and a confidence score $c \in [0, 1]$ indicating certainty.

Agent nodes represent AI agents with persistent positions in the space. An agent’s position is computed as the weighted centroid of its recently accessed or created cells:

$$\mathbf{e}_{\text{agent}} = \frac{\sum_{i=1}^n w_i \cdot \mathbf{e}_i}{\|\sum_{i=1}^n w_i \cdot \mathbf{e}_i\|} \quad \text{where} \quad w_i = s_i \cdot e^{-\lambda \cdot \Delta t_i} \quad (2)$$

with s_i the salience of cell i , Δt_i the time since last access, and $\lambda = 0.1$ the decay rate. As the agent’s work shifts topics, its position drifts through the vector space, naturally entering and leaving different topological neighborhoods.

Scene nodes are topic centroids produced by consolidation—cluster centers that summarize a coherent knowledge area (e.g., “Authentication — OAuth2”). They occupy Layer 1 or higher.

Beacon nodes are standing subscriptions placed at fixed positions. A security review agent can place a beacon at the “security risks” position; any gossip passing through that region activates the beacon and delivers to the agent’s inbox, regardless of the agent’s current position.

3.2 Primitive 2: Topology Edges

Edges define communication channels. They exist implicitly in the HNSW graph (managed by pgvector for search) and explicitly in a topology edge table:

$$\text{edge} = (\text{src}, \text{tgt}, \tau_e, \omega, b) \quad (3)$$

where $\tau_e \in \{\text{semantic}, \text{causal}, \text{subscription}, \text{promoted}\}$ is the edge type, $\omega \in (0, 1]$ is the connection weight, and b is the bandwidth (gossip capacity). The hop cost of traversing an edge is ω^{-1} : strong connections are cheap to traverse, weak connections are expensive.

3.3 Primitive 3: Memory Diffs

The unit of gossip is a *memory diff*—a change notification that propagates through the topology:

$$\text{diff} = (\text{origin}, \tau_d, \mathbf{e}_d, E, s, h, h_{\max}, \mathcal{P}, \text{TTL}) \quad (4)$$

where $\tau_d \in \{\text{created}, \text{updated}, \text{superseded}, \text{contradiction}, \text{pattern}, \text{promoted}\}$, $\mathbf{e}_d$ is the diff’s embedding, E is the remaining energy, s is salience, h is the current hop count, $h_{\max}$ is the maximum hops, $\mathcal{P}$ is the set of visited node IDs, and TTL is the time-to-live.

3.4 Multi-Layer Abstraction

The topology organizes knowledge across three abstraction layers, each with its own HNSW index:

- **Layer 0** (raw cells): Dense connections, all knowledge nodes. HNSW parameters: $M = 24$, efConstruction = 128.
- **Layer 1** (scene-level): Scene centroids and promoted patterns. Medium-range connections. $M = 16$, efConstruction = 100.
- **Layer 2** (project-level): Architectural decisions and constraints. Long-range connections. $M = 8$, efConstruction = 64.

This mirrors HNSW’s own hierarchical structure but at the *semantic* level rather than the geometric level: Layer 0 contains raw observations, Layer 1 contains patterns, and Layer 2 contains principles.

4 The Gossip Protocol

4.1 Propagation Algorithm

When a knowledge change occurs, the gossip engine creates a memory diff and executes Algorithm 1.

Algorithm 1 Gossip Propagation

Require: diff δ , current node v

```
1: if  $E(\delta) \leq 0$  or  $h(\delta) \geq h_{\max}(\delta)$  or  $v \in \mathcal{P}(\delta)$  then
2:   return
3: end if
4: DELIVER( $\delta, v$ ) {Record delivery; add to inbox if agent/beacon}
5:  $\mathcal{P}(\delta) \leftarrow \mathcal{P}(\delta) \cup \{v\}$ ;  $h(\delta) \leftarrow h(\delta) + 1$ 
6:  $\mathcal{N} \leftarrow \text{HNSWNEIGHBORS}(v, \ell(\delta), K)$  {K nearest at current layer}
7: for each  $u \in \mathcal{N} \setminus \mathcal{P}(\delta)$  do
8:    $r \leftarrow \cos(\mathbf{e}_\delta, \mathbf{e}_u)$  {Semantic relevance}
9:    $\iota \leftarrow \text{INTEREST}(u, \delta)$  {Node's declared interest}
10:   $\sigma \leftarrow 0.6 \cdot r + 0.3 \cdot \iota + 0.1 \cdot s(\delta)$  {Combined score}
11:   $c \leftarrow \omega(v, u)^{-1}$  {Hop cost}
12:  if  $\sigma < \sigma_{\min}$  or  $E(\delta) < c$  then
13:    continue
14:  end if
15:   $\delta' \leftarrow \text{COPY}(\delta)$ ;  $E(\delta') \leftarrow E(\delta) - c$ 
16:  PROPAGATE( $\delta', u$ )
17: end for
18: if SHOULD_PROMOTE( $\delta, v$ ) then
19:   $\delta_p \leftarrow \text{PROMOTE}(\delta, \ell(v) + 1)$ 
20:   $u_p \leftarrow \text{FINDLAYERENTRY}(v, \ell(v) + 1)$ 
21:  PROPAGATE( $\delta_p, u_p$ )
22: end if
```

The critical difference from standard gossip protocols is line 8: neighbor selection follows HNSW's *semantic* topology rather than random peer selection. This ensures knowledge reaches semantically relevant agents before irrelevant ones.

4.2 Energy Model

Each diff's initial energy determines its propagation radius:

$$E_0 = E_{\text{base}} \cdot \sqrt{s} \cdot \mu(\tau_d) \quad (5)$$

where E_{base} is a configurable base energy (default 10.0), s is salience, and $\mu(\tau_d)$ is a type-dependent multiplier:

Table 1: Energy multipliers by knowledge type.

Type	μ	Typical E_0	Propagation Scope
Decision	2.0	~ 19	Topology-wide
Constraint	2.0	~ 19	Topology-wide
Risk	1.5	~ 14	Most of topology
Pattern	1.5	~ 14	Most of topology
Lesson	1.2	~ 11	Broad regional
Fact	1.0	~ 9	Regional
Preference	0.8	~ 7	Local
Context	0.5	~ 5	Neighborhood
Task	0.3	~ 3	Immediate vicinity

The square root in Equation 5 compresses the salience range, preventing extreme values from dominating propagation. The type multiplier encodes domain knowledge: architectural decisions should propagate broadly; task-specific details should stay local.

4.3 Layer Promotion

Knowledge that receives confirmation from multiple nodes at the same layer can be *promoted* to a higher layer, gaining access to longer-range connections.

Layer 0 $\rightarrow$ 1: The diff has been delivered to ≥ 5 nodes within the same scene, with no contradictions flagged. Eligible types: decision, pattern, constraint, lesson.

Layer 1 $\rightarrow$ 2: The diff has been delivered to ≥ 3 scenes without contradictions. Salience threshold: $s \geq 0.8$. Eligible types: decision, constraint.

Upon promotion, the system creates a new node at the higher layer, grants the diff a fresh energy bonus ($E_{\text{bonus}} = 5.0$), and propagation continues through the higher layer’s longer-range connections. This is the mechanism for *emergent consensus*: local agreement promotes to regional agreement, which promotes to global knowledge, without centralized decision-making.

4.4 Anti-Entropy

Gossip provides probabilistic delivery. Anti-entropy sweeps provide the guarantee. Periodically, the system compares state between topological neighbor pairs using vector clocks and reconciles differences by forwarding missing diffs. This follows the two-phase pattern of Demers et al. [1987]: rumor mongering for speed, anti-entropy for completeness.

5 Emergent Behaviors

The combination of HNSW topology, semantic gossip, energy-based attenuation, and layer promotion produces four emergent behaviors.

5.1 Automatic Knowledge Routing

When an agent stores a decision about API rate limiting, the created diff propagates outward through Layer 0 neighbors. It reaches the code agent (positioned near API topics by virtue of recent work) and the review agent’s “breaking change” beacon, but not the test agent positioned in a distant region. No subscription was configured; the topology determined relevance.

This property follows from the HNSW graph’s small-world structure: nodes that are semantically close are topologically close, so gossip naturally reaches relevant agents first. The energy model ensures the diff attenuates before reaching irrelevant regions.

5.2 Contradiction Detection

When two cells with high semantic similarity ($\cos(\mathbf{e}_a, \mathbf{e}_b) > \theta_{\text{sim}}$) but opposing content are stored in proximity, gossip delivers both signals to the same neighborhood. The system detects the divergence and automatically generates a contradiction diff with bonus energy ($1.5\times$) for broad propagation.

This works because conflicting knowledge occupies nearby positions in the embedding space. The topology makes contradiction detection a *geometric* property rather than an algorithmic one, requiring no explicit contradiction-checking logic.

5.3 Pattern Crystallization

When multiple agents independently store similar observations (e.g., three agents reporting different instances of batch processing failures), these cells cluster in Layer 0. The consolidation worker detects the cluster density, extracts a generalized pattern via LLM, and the pattern cell—with high salience—promotes to Layer 1 via the promotion mechanism, propagating broadly.

This follows the same positive-feedback dynamics as pheromone accumulation in ant colony optimization [Dorigo and Stützle, 2004]: individual signals reinforce each other through proximity and repetition, producing emergent collective knowledge.

5.4 Organic Knowledge Decay

Cells that are not accessed, confirmed, or referenced by gossip undergo natural energy decay. Their salience diminishes via:

$$s(t) = s_0 \cdot 0.99^{\Delta t} \tag{6}$$

where Δt is days since last access. Cells below a configurable threshold stop participating in gossip propagation but remain searchable in Layer 0 for direct queries. If a new decision supersedes a stale one, the superseded diff propagates through the region, informing affected agents.

6 Theoretical Analysis

6.1 Propagation Guarantees

Proposition 1 (Propagation Reach). *In an HNSW graph with N knowledge nodes and connectivity parameter M , a memory diff with sufficient energy reaches all nodes in the same topological neighborhood in $O(\log N)$ gossip rounds.*

Proof sketch. The HNSW graph satisfies the small-world property [Malkov and Ponomarenko, 2016]: average path length scales as $O(\log N)$ and the clustering coefficient is bounded below by a positive constant. By the epidemic spreading results of Moore and Newman [2000], information propagation on such graphs completes in $O(\log N)$ rounds with fanout $f \geq 2$. Since our propagation algorithm selects the K highest-scoring neighbors at each hop (with K configurable, default 10), the effective fanout exceeds the threshold for rapid propagation. Energy attenuation bounds the *scope* of propagation but not its speed within the relevant subgraph. $\square$

Proposition 2 (Energy-Bounded Scope). *A diff with initial energy E_0 on a graph with average edge weight $\bar{\omega}$ propagates to at most $E_0 \cdot \bar{\omega}$ nodes.*

Proof sketch. Each hop costs at least $\bar{\omega}^{-1}$ energy (from the minimum-cost edge). With E_0 total energy, the maximum number of hops is $E_0 \cdot \bar{\omega}$. Since each hop visits one new node (by cycle prevention), the bound follows. In practice, the branching factor means fewer nodes are reached in a deep chain but more in a broad wavefront, preserving the bound in expectation. $\square$

6.2 Convergence of Layer Promotion

The layer promotion mechanism produces a form of distributed consensus. For a knowledge fragment to reach Layer 2, it must be confirmed by ≥ 5 nodes in a single scene (Layer 0 $\rightarrow$ 1) and then by ≥ 3 scenes (Layer 1 $\rightarrow$ 2), all without contradiction. This is structurally analogous to Byzantine fault tolerance with $f < n/3$: a minority of dissenting nodes cannot prevent valid promotions, but a significant contradiction blocks them.

Olfati-Saber et al. [2007] showed that consensus dynamics on small-world graphs converge in $\Theta(\log N)$ time, governed by the algebraic connectivity (second-smallest eigenvalue of the graph Laplacian). Since HNSW graphs have high algebraic connectivity (short path lengths and dense local connections), we expect rapid convergence of the promotion mechanism.

6.3 Self-Organization Properties

Based on the results of Buehler [2025], we conjecture that HyphaeDB’s topology will develop several self-organizing properties over time:

1. **Emergent hubs:** Frequently-confirmed knowledge cells will accumulate connections and become high-centrality nodes, serving as routing shortcuts.
2. **Modular structure:** Topically distinct regions will develop clear boundaries, with higher internal connectivity than cross-boundary connectivity.
3. **Bridge nodes:** Interdisciplinary knowledge (e.g., security implications of architectural decisions) will occupy positions linking otherwise disconnected modules.

4. **Scale-free degree distribution:** The combination of preferential attachment (popular cells attract more gossip) and the HNSW construction process will produce power-law degree distributions in the explicit edge table.

These conjectures are testable and form the basis for empirical validation in future work.

7 Implementation

7.1 Technology Stack

The reference implementation uses PostgreSQL with the pgvector extension [Katz et al., 2024], which provides native HNSW indexing. This choice provides ACID transactions, SQL joins for structured queries, recursive CTEs for graph traversal, and HNSW vector search in a single deployment. pgvector 0.8.0 introduced iterative index scans, which automatically re-scan the HNSW index when filtered queries discard too many neighbors—critical for queries combining vector similarity with metadata filters.

The API layer is TypeScript with Express, exposing both a REST API and a Model Context Protocol (MCP) server for native integration with MCP-compatible clients (Claude Code, Cursor, Cline).

7.2 Schema Design

The system uses per-layer partial HNSW indexes:

```
CREATE INDEX idx_nodes_layer0_embedding
ON knowledge_nodes USING hnsw
(embedding vector_cosine_ops)
WITH (m = 24, ef_construction = 128)
WHERE layer = 0;
```

```
CREATE INDEX idx_nodes_layer1_embedding
ON knowledge_nodes USING hnsw
(embedding vector_cosine_ops)
WITH (m = 16, ef_construction = 100)
WHERE layer = 1;
```

Runtime configuration sets `hnsw.ef_search = 100` (up from the default 40) and enables `hnsw.iterative_scan = relaxed_order` for filtered queries. Topology neighbors are retrieved via cosine distance ordering with a `LIMIT` clause, leveraging pgvector’s HNSW index for $O(\log N)$ neighbor lookup.

7.3 Reference Deployment: Swarm-Driven Development

The reference deployment uses Swarm-Driven Development (SDD), a methodology for orchestrating multi-agent software development teams. In SDD, specialized agents (specification, code, review, test) collaborate across multi-session development workflows. HyphaeDB provides their shared persistent memory with automatic knowledge routing.

An orchestrator hook (`withMemory()`) wraps each agent function with automatic recall before execution and extraction after. Before an agent runs, the hook queries the mesh for task-relevant context, active decisions, constraints, and Jira-scoped knowledge, injecting the results into the agent’s system prompt. After the agent completes, the hook extracts structured memory cells from the output via LLM and stores them in the topology. The agent itself is unaware of the memory system; the hook makes memory transparent.

Ten MCP tools expose the mesh to agents: `sdd_memory_recall` (auto-recall context), `sdd_memory_query` (multi-mode search), `sdd_memory_store` (write cell), `sdd_memory_extract` (LLM extraction), `sdd_memory_decisions` (decision log), `sdd_memory_lessons` (lessons and risks), `sdd_memory_scenes` (list scenes), `sdd_memory Consolidate` (trigger consolidation), and session management tools for provenance tracking.

8 Competitive Analysis

Table 2 compares HyphaeDB with existing vector databases and agent memory frameworks across 13 capabilities.

Table 2: Capability comparison across memory infrastructure.

Capability	Pinecone	Weaviate	Mem0	HyphaeDB
Vector search	✓	✓	✓	✓
Structured metadata	✓	✓	Partial	✓
Graph relationships	—	Partial	—	Native
Multi-agent awareness	—	—	—	Core
Knowledge propagation	—	—	—	Novel
Automatic routing	—	—	—	Novel
Contradiction detect	—	—	—	Emergent
Pattern emergence	—	—	—	Emergent
Layer promotion	—	—	—	Novel
Agent positioning	—	—	—	Novel
Standing beacons	—	—	—	Novel
Energy propagation	—	—	—	Novel
Self-hosted (Postgres)	—	✓	✓	✓

Nine of thirteen capabilities are unique to HyphaeDB. The existing systems—Pinecone (\$138M raised, \$750M valuation), Weaviate (\$50–67M raised), Mem0 (\$24M raised), Milvus/Zilliz (\$113M raised)—all operate as passive store-and-retrieve engines. They represent the infrastructure for the *previous* paradigm (search); HyphaeDB represents the infrastructure for the *next* paradigm (coordination).

9 Discussion

9.1 Design Tradeoffs

Synchronous vs. batched gossip. The current design propagates diffs synchronously on write. This provides low-latency delivery but risks cascade storms when many cells are created simultaneously. An alternative is batched propagation on a periodic sweep (e.g., every 100ms), which adds latency but provides predictable throughput. The optimal choice likely depends on the deployment’s write rate and may benefit from adaptive switching.

Energy model calibration. The square-root salience multiplier and type-specific multipliers in Equation 5 are initial values that require empirical validation against real agent workflows. Overly generous energy budgets waste bandwidth; overly conservative budgets starve agents of relevant knowledge.

Contradiction resolution. The system detects contradictions automatically but does not resolve them. Resolution options include human-in-the-loop arbitration, agent voting (weighted by salience of the contributing agents), recency-wins policies, and salience-wins policies. The appropriate strategy likely varies by domain and should be configurable per project.

9.2 Limitations and Future Work

Empirical validation. The emergent behaviors described in Section 5 are theoretically motivated but not yet empirically validated. A critical next step is deploying HyphaeDB in production multi-agent workflows and measuring: (a) proportion of relevant knowledge that reaches agents without explicit query; (b) time-to-detection for contradictions; (c) accuracy of crystallized patterns; (d) correspondence between layer promotion and human expert agreement.

Cross-project knowledge transfer. The current design operates within a single project. Extending to cross-project knowledge transfer—where patterns learned in Project A propagate to Project B via a “meta-mesh”—requires solving relevance filtering across different semantic domains and establishing trust boundaries.

Embedding model migration. Changing embedding models invalidates all vector positions. A migration strategy involving dual-write during transition, lazy re-embedding on read, and batch background re-embedding is needed for production deployments.

Adversarial resilience. The energy model limits blast radius of any single agent’s output, but a malicious agent could inject high-salience diffs to propagate misinformation. Trust scoring based on agent provenance and confirmation history may be necessary for adversarial environments.

Formal convergence proofs. While we provide propagation guarantees (Section 6), formal proofs of convergence for the layer promotion mechanism, contradiction detection completeness, and pattern crystallization conditions remain open.

10 Conclusion

We have presented HyphaeDB, an agent-native memory infrastructure that reinterprets the HNSW graph topology as a communication fabric for multi-agent systems. By combining navigable small-world graph structure with gossip-based knowledge propagation and energy-based attenuation, HyphaeDB enables memory that actively participates in agent coordination rather than passively serving queries.

The system introduces several novel constructs: agent positioning via weighted centroids of accessed knowledge, multi-layer abstraction with promotion via emergent consensus, standing beacons for persistent subscriptions, and an energy model that automatically calibrates propagation scope to knowledge importance. Theoretical analysis grounds these constructs in small-world network theory, epidemic broadcast models, and swarm intelligence, predicting $O(\log N)$ propagation time, energy-bounded scope, and emergent self-organization.

To our knowledge, HyphaeDB is the first system to combine HNSW topology with gossip-based knowledge propagation for multi-agent coordination. The reference implementation on PostgreSQL with pgvector is

available as open-source software at <https://github.com/hyphae-db>, with deployment demonstrated in Swarm-Driven Development, a multi-agent software engineering methodology.

The agent infrastructure market is at an inflection point: enterprise adoption is accelerating from 5% to 40% of applications in a single year, yet 50% of agents operate in isolated silos. HyphaeDB addresses this gap not by adding features to passive storage, but by building infrastructure where the topology itself is the coordination mechanism. Memory is the medium. Intelligence is the message.

References

James Aspnes and Gauri Shah. Skip graphs. In *Proceedings of the 14th Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 384–393, 2003.

Florent Bénézit et al. Revisiting gossip protocols: A vision for emergent coordination in agentic multi-agent systems. *arXiv preprint arXiv:2508.01531*, 2025.

Markus J Buehler. Agentic deep graph reasoning yields self-organizing knowledge networks. *Machine Learning: Science and Technology*, 2025.

Ian Clarke, Oskar Sandberg, Matthew Toseland, and Vilhelm Verendel. Private communication through a network of trusted connections: The dark Freenet. In *Workshop on Privacy in the Electronic Society*, 2005.

Abhinandan Das, Indranil Gupta, and Ashish Motivala. SWIM: Scalable weakly-consistent infection-style process group membership protocol. In *Proceedings of the International Conference on Dependable Systems and Networks*, pages 303–312. IEEE, 2002.

Alan Demers, Dan Greene, Carl Hauser, Wes Irish, John Larson, Scott Shenker, Howard Sturgis, Dan Swinehart, and Doug Terry. Epidemic algorithms for replicated database maintenance. In *Proceedings of the Sixth Annual ACM Symposium on Principles of Distributed Computing*, pages 1–12. ACM, 1987.

Marco Dorigo and Thomas Stützle. *Ant Colony Optimization*. MIT Press, 2004.

Gartner. Gartner predicts 40% of enterprise apps will feature task-specific AI agents by 2026. <https://www.gartner.com/en/newsroom/press-releases/2025-08-26-gartner-predicts-40-percent-of-enterprise-apps-will-feature-task-specific-ai-agent>, 2025.

Grand View Research. AI agents market size and share – industry report, 2033. <https://www.grandviewresearch.com/industry-analysis/ai-agents-market-report>, 2025.

Pierre-Paul Grassé. La reconstruction du nid et les coordinations interindividuelles chez *Bellicositermes Natalensis* et *Cubitermes* sp. *Insectes Sociaux*, 6(1):41–80, 1959.

Márk Jelasity and Özalp Babaoglu. T-Man: Gossip-based fast overlay topology construction. *Computer Networks*, 53:2321–2339, 2009.

Jonathan Katz et al. pgvector: Open-source vector similarity search for Postgres. <https://github.com/pgvector/pgvector>, 2024.

David Kempe, Jon Kleinberg, and Alan Demers. Spatial gossip and resource location protocols. In *Proceedings of the 33rd Annual ACM Symposium on Theory of Computing*, pages 163–172. ACM, 2001.

- Jon M Kleinberg. Navigation in a small world. *Nature*, 406(6798):845–845, 2000.
- LangChain. LangMem SDK for agent long-term memory. <https://blog.langchain.com/langmem-sdk-launch/>, 2025.
- João Leitão, José Pereira, and Luís Rodrigues. HyParView: A membership protocol for reliable gossip-based broadcast. In *Proceedings of the International Conference on Dependable Systems and Networks*, pages 419–429. IEEE, 2007a.
- João Leitão, José Pereira, and Luís Rodrigues. Epidemic broadcast trees. In *Proceedings of the 26th IEEE International Symposium on Reliable Distributed Systems*, pages 301–310. IEEE, 2007b.
- Yu A Malkov and D A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 42(4): 824–836, 2020. doi: 10.1109/TPAMI.2018.2889473.
- Yury A Malkov and Alexander Ponomarenko. Growing homophilic networks are natural navigable small worlds. *PLoS ONE*, 11(8):e0158162, 2016.
- Gurmeet Singh Manku, Mayank Bawa, and Prabhakar Raghavan. Symphony: Distributed hashing in a small world. In *Proceedings of the 4th USENIX Symposium on Internet Technologies and Systems*, pages 10–10, 2003.
- MarketsandMarkets. AI agents market size, share and trends – growth analysis, forecast to 2030. <https://www.marketsandmarkets.com/Market-Reports/ai-agents-market-15761548.html>, 2025.
- Cristopher Moore and Mark E J Newman. Epidemics and percolation in small-world networks. *Physical Review E*, 61(5):5678, 2000.
- Adrien Munyampirwa, Vyas Lakshman, and Sriram Srinivasan. The hub highway hypothesis: Understanding hnsw graph structure through hub connectivity. *arXiv preprint arXiv:2407.01573*, 2024.
- Reza Olfati-Saber, J Alex Fax, and Richard M Murray. Consensus and cooperation in networked multi-agent systems. *Proceedings of the IEEE*, 95(1):215–233, 2007.
- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G Patil, Ion Stoica, and Joseph E Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- Craig W Reynolds. Flocks, herds and schools: A distributed behavioral model. In *Proceedings of the 14th Annual Conference on Computer Graphics and Interactive Techniques*, pages 25–34. ACM, 1987.
- Salesforce. Multi-agent adoption to surge 67% by 2027 as enterprises race toward agentic transformation. <https://www.salesforce.com/news/stories/connectivity-report-announcement-2026/>, 2026.
- Duncan J Watts and Steven H Strogatz. Collective dynamics of ‘small-world’ networks. *Nature*, 393(6684): 440–442, 1998.
- Wujiang Xu et al. A-Mem: Agentic memory for LLM agents. *Advances in Neural Information Processing Systems*, 38, 2025.
- Deshraj Yadav, Prateek Shukla, Gunjan Chhablani, and Raghav Rawat. Mem0: Building production-ready ai agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Wei Zhang et al. Memory in LLM-based multi-agent systems: Mechanisms, challenges, and collective intelligence. *TechRxiv Preprint*, 2025. doi: 10.36227/techrxiv.174053233.55786655/v1.